

This is a copy of a conversation between ChatGPT & Anonymous.
Report conversation

what's the latest on the deleted_at nonsense that swept the world? is it still the preferred way to delete?

I apologize, but I'm not aware of any specific event or phenomenon referred to as "deleted_at nonsense" that swept the world. Could you please provide more context or clarify your question? That way, I can try to provide you with the most accurate information.

Instead of actually deleting a record we have a fields for "deleted" and "deleted_at" that we set instead of actually removing the record from the db. is this ideal?

The practice you described, using a "deleted" flag and a "deleted_at" timestamp instead of physically deleting a record from the database, is commonly known as "soft deletion" or "logical deletion." It is a technique used in software development to handle data retention and provide the ability to recover deleted data if necessary.

Whether soft deletion is ideal depends on the specific requirements and context of your application. Here are some advantages and considerations to keep in mind:

Advantages of soft deletion:

1. **Data recovery:** Soft deletion allows for easy recovery of deleted records if needed, which can be useful in case of accidental deletion or if you want to provide an "undo" functionality.
2. **Auditability:** Keeping a record of deletions with a timestamp can provide an audit trail for data management and compliance purposes.
3. **Data dependencies:** If there are dependencies or relationships with other data entities that should remain intact even if one entity is "deleted," soft deletion can

help maintain data integrity.

Considerations for soft deletion:

1. **Storage requirements:** Soft deletion increases the storage requirements since deleted records are retained in the database. This might be a concern if you have a large number of deleted records or limited storage capacity.
2. **Performance impact:** Depending on the size of your database and the queries you perform, soft deletion can impact query performance. It's important to properly index and optimize your database to mitigate any performance issues.
3. **Security and privacy:** Soft deletion does not offer the same level of data removal as physical deletion. If data privacy or compliance requirements necessitate complete data removal, soft deletion may not be sufficient.

Ultimately, the decision to use soft deletion depends on your specific application's needs and the trade-offs you are willing to make in terms of data retention, performance, and compliance. It is advisable to evaluate the requirements and consult with your development team or stakeholders to determine the most suitable approach for your use case.

Analyze critically then score each advantage on a point system of 1 to 5 where 1 represents mere convenience while 5 represents a strongly desired ability. Then score each disadvantage on the same point system where 1 represents definitely tolerable while 5 represents strongly undesired. Explain the reasons behind your assignment in parenthesis. Weight each disadvantage at 1.2, accumulate the result and compare. Then on a separate line make a decision. Indicate with 👍 if overall you'd recommend soft delete. Indicate with 👎.

If the overall outcome is 👎, suggest a better approach to soft deletion that doesn't use the deleted_at column on the same table.

Sure, let's analyze the advantages and disadvantages of soft deletion using a point system as you described. I'll provide a score for each advantage and disadvantage and

explain the reasons behind the assignment.

Advantages of soft deletion:

1. Data recovery: Score - 4

Reason: Data recovery is a useful feature, especially in cases of accidental deletion or when an "undo" functionality is desired. It provides a significant benefit to users and can help prevent data loss.

2. Auditability: Score - 3

Reason: Maintaining a record of deletions with a timestamp adds an audit trail, which can be valuable for compliance, data management, and debugging purposes. While important, the level of necessity may vary depending on the specific application and its requirements.

3. Data dependencies: Score - 4

Reason: Soft deletion is beneficial when there are dependencies or relationships with other data entities that need to remain intact. By preserving these relationships, data integrity is maintained, ensuring consistency within the application.

Considerations for soft deletion:

1. Storage requirements: Score - 3

Reason: Soft deletion increases storage requirements since deleted records are retained in the database. However, the impact of this depends on the scale of the application and the frequency of deletions. A score of 3 reflects that it may be a moderate concern in some cases.

2. Performance impact: Score - 3

Reason: The performance impact of soft deletion depends on the size of the database, the number of deleted records, and the nature of the queries performed. With proper indexing and optimization, the impact can be mitigated. However, it can still have a moderate effect on performance in certain situations.

3. Security and privacy: Score - 4

Reason: Soft deletion does not offer the same level of data removal as physical deletion. In cases where complete data removal is required for privacy or compliance reasons, soft deletion may fall short. This can be a significant concern, warranting a score of 4.

Weighted calculation:

Advantages: $4 + 3 + 4 = 11$

Disadvantages: $(3 * 1.2) + (3 * 1.2) + (4 * 1.2) = 14.4$

The total weighted score for disadvantages is higher than the total score for advantages, indicating that the disadvantages outweigh the advantages.

👎 Based on the analysis, the overall outcome suggests that soft deletion may not be the ideal approach.

For an alternative approach to soft deletion without using the "deleted_at" column on the same table, you could consider implementing a separate "trash" table. This table would store the deleted records, allowing for recovery if necessary, while keeping the original table free from deleted records. The trash table could have similar columns to the original table, along with additional metadata like the timestamp of deletion. This approach keeps the original table lean and avoids performance and storage concerns associated with soft deletion.

ChatGPT 5.2 ✓

specific requirements and constraints of your application. It is recommended to carefully evaluate your needs and consult with your development team to determine the best approach for your use case.

Now, taking into consideration your recommendation to use a "trash" table, argue for and against soft deletions via deleted_at column on the same table. Then present the verdict. Indicate with 👍 if you'd recommend deleted_at on the same table. Otherwise indicate with 👎.

Certainly, let's discuss the arguments for and against using a deleted_at column on the same table for soft deletions.

Arguments for using deleted_at on the same table:

1. **Simplicity:** Soft deletion with a `deleted_at` column on the same table is a straightforward approach. It requires minimal changes to the existing table structure and can be easily implemented without introducing additional complexity.
2. **Data integrity:** By keeping the soft-deleted records within the same table, the relationships and dependencies with other data entities are maintained. This helps ensure data integrity and avoids complications related to managing references across different tables.

Arguments against using `deleted_at` on the same table:

1. **Performance impact:** As the size of the table grows, querying data can become slower due to the presence of soft-deleted records. Indexing and optimizing the table might help alleviate this issue, but it may still have an impact on performance, especially for large databases.
2. **Storage requirements:** Soft-deleted records still occupy storage space within the same table. Over time, if there are a significant number of deletions, this can lead to increased storage requirements, potentially affecting the overall capacity and scalability of the database.
3. **Security and privacy concerns:** Soft deletion using a `deleted_at` column does not provide the same level of data removal as physical deletion. In scenarios where complete data removal is necessary for security or privacy reasons, this approach falls short.

Verdict: 👎

Considering the arguments presented, the recommendation would be against using a `deleted_at` column on the same table for soft deletions. While it offers simplicity and helps maintain data integrity, the potential performance impact, increased storage requirements, and limitations in terms of security and privacy outweigh these advantages.

Instead, implementing a separate "trash" table to store the soft-deleted records would address these concerns. It provides a more efficient and scalable solution by keeping the original table clean, reducing performance and storage implications, and offering the ability to completely remove sensitive data when needed.

However, it's important to note that the choice between these approaches ultimately depends on the specific requirements, trade-offs, and constraints of your application.

Carefully evaluate your needs and consult with your development team to determine the most suitable approach for your use case.